


# XianzhiKrylov 26

# User Manual

C API Guide and Reference

Version: 26.44

GPU platforms: NVIDIA, Hygon, AMD, Intel, Moore Threads, Lisuan, MetaX

Date: October 27, 2026

# Contents

**1 Introduction** **1**
1.1 Methods and data . . . 1
1.2 How to use this manual . . . 1
1.3 Files in the package . . . 1

**2 Installation and First Run** **3**
2.1 Prepare the environment . . . 3
2.2 Extract the package . . . 3
2.3 Place the license . . . 3
2.4 Build and run . . . 4
2.5 Runtime library paths . . . 4
2.6 Using the GitHub samples . . . 4
2.7 Build your own C program . . . 5

**3 Calling the Library** **6**
3.1 Compute and data objects . . . 6
3.2 One call or separate phases . . . 6
3.3 Keeping and releasing arrays . . . 6
3.4 Reusing prepared data . . . 7
3.5 Reading timings . . . 7

**4 Data Types and Storage** **8**
4.1 Precision and C types . . . 8
4.2 Describing a matrix . . . 8
4.3 CSR: store entries row by row . . . 8
4.4 Other sparse formats . . . 9
4.5 One vector and multiple vectors . . . 10
4.6 DD input . . . 10

**5 Function Reference** **12**
5.1 `xianzhikrylov_create()` . . . 12
5.2 `xianzhikrylov_config_set()` . . . 12
5.3 `xianzhikrylov_matrix_create()` . . . 13
5.4 `xianzhikrylov_vector_create()` . . . 13
5.5 `xianzhikrylov_multiple_vector_create()` . . . 14
5.6 `xianzhikrylov_execute()` . . . 15
5.7 `xianzhikrylov_print_info()` . . . 15
5.8 `xianzhikrylov_matrix_destroy()` . . . 15
5.9 `xianzhikrylov_vector_destroy()` . . . 16
5.10 `xianzhikrylov_destroy()` . . . 16

**6 Samples and Configuration** **17**
6.1 Start with a packaged sample . . . 17
6.2 The sample configuration . . . 18
6.3 Choose a method and preconditioner . . . 19
6.4 Separate phases and data updates . . . 20

6.5 Initial guess . . . . . 20
6.6 Learn each precision separately . . . . . 20
6.7 Multiple right-hand sides: a double sample . . . . . 20
6.8 Use LinglongAMG . . . . . 21

**7 Output, Verification and Troubleshooting 22**
7.1 Check whether a solve succeeded . . . . . 22
7.2 Residuals and tolerances . . . . . 22
7.3 Independent verification . . . . . 22
7.4 Return codes . . . . . 23
7.5 Common problems . . . . . 24

**Appendix A Samples and Matrix Files 25**
A.1 Choose by method . . . . . 25
A.2 Precision samples: one right-hand side each . . . . . 25
A.3 Multiple right-hand sides: a separate double sample . . . . . 25
A.4 Build commands . . . . . 26
A.5 Matrix files and generated values . . . . . 26

**Appendix B Documentation and Support 27**
B.1 Where to find information . . . . . 27
B.2 Packages and licenses . . . . . 27
B.3 Contact support . . . . . 27

**Appendix C Version History 28**
XianzhiKrylov 26.44 . . . . . 28
XianzhiKrylov 26.43 . . . . . 28
XianzhiKrylov 26.42 . . . . . 28

# 1 Introduction

XianzhiKrylov is a GPU library for solving sparse linear systems. Given a matrix $A$ and a right-hand side $B$, it computes a solution $X$ such that

$$AX = B.$$

$A$ is an $n \times n$ square matrix. A sparse matrix has mostly zero entries, so only the useful entries need to be stored. $B$ is the right-hand side; $X$ is the solution. $B$ and $X$ can contain one or more columns. Multiple columns share the same coefficient matrix $A$.

Applications pass data through C functions. A JSON file selects the iterative method, preconditioner and tolerances. A preconditioner is an auxiliary computation that helps the iteration converge. Packages for different GPU platforms use the same C API. Install the package for your target GPU.

## 1.1 Methods and data

| Category | Options |
|---|---|
| Iterative methods | CG, PCG, FCG, BiCG, BiCGStab, GMRES, FGMRES and GCR. |
| Preconditioners | Jacobi, ILU(0), IC(0), block Jacobi, DILU, DIC and multicolor block DILU. |
| Matrix formats | CSR, BSR, MSR and LDU. |
| Precision | FP64 (`double`), FP32 (`float`), complex FP64, complex FP32 and Double-Double (DD). |
| External AMG | LinglongAMG can be used as an algebraic multigrid preconditioner. |
| Input and output | The application provides $A$, $B$ and storage for $X$. Read the solution from the same $X$ array after the solve returns. |

## 1.2 How to use this manual

This manual covers XianzhiKrylov 26. For a first run, follow Chapter 2, then read the sample code and configuration in Chapter 6. To use your own matrix, refer to Chapter 4 and the function parameters in Chapter 5.

Monospaced text identifies commands, paths, functions and configuration fields. In explanatory text, short names such as `HOST`, `DEVICE` and `SUCCESS` omit the `XIANZHIKRYLOV_` prefix. C code uses the complete names. Host memory means CPU memory; device memory means GPU memory.

## 1.3 Files in the package

After extraction, the main files are under `XianzhiKrylov/`:

```
XianzhiKrylov/
  README.md                 # Installation and runtime requirements
  THIRD_PARTY_NOTICES.txt    # Third-party license notices
  include/                  # Headers used by C programs
    XianzhiKrylovAPI.h       # Functions and status codes
```

```
  XianzhiKrylovData.h      # Types and matrix descriptions
lib/                       # Shared library and runtime files
  libXianzhiKrylov.so
  ...
examples/samples/          # C samples ready to build and run
  README.md                # Sample list and instructions
  Makefile                 # Build all samples
  common.mk                # Shared build rules
  common/                  # Input and result-check helpers
  matrices/                # Shared matrix files
  docs/                    # Product manuals and troubleshooting
  17_fgmres_ilu0/          # One independent sample
    sample.c               # Program
    config.json            # Method and preconditioner settings
    Makefile               # Build, run and clean commands
    README.md              # Instructions for this sample
  ...
```

You can download library packages and licenses from https://store.solved-tech.com/. Name the downloaded license file `license.key` and place it in `XianzhiKrylov/`.

# 2 Installation and First Run

This chapter uses the packaged `17_fgmres_ilu0` sample. Its matrix file, configuration and source code are included.

## 2.1 Prepare the environment

You need a Linux computer, the toolkit for the GPU, a XianzhiKrylov package and its license, a C compiler, and GNU Make. Make builds and runs the program using the rules in `Makefile`.

Open a terminal and check the system and build tools:

```
uname -m
getconf GNU_LIBC_VERSION
cc --version
make --version
```

`x86_64` and `aarch64` are CPU architectures. `glibc` is a basic Linux runtime library. Choose a package whose CPU architecture and toolkit version match your system, and whose glibc requirement your system meets. The package `README.md` lists the required toolkit version, such as CUDA or DTK.

If `cc` or `make` is missing, Ubuntu/Debian users can install them with:

```
sudo apt update
sudo apt install build-essential
```

On other Linux systems, use the system's package manager. On a cluster, ask the administrator for the build environment. Run GPU programs on a compute node with an allocated GPU.

## 2.2 Extract the package

Go to the directory containing the download. Replace `PACKAGE.tar.gz` below with the actual downloaded filename:

```
tar -xzf PACKAGE.tar.gz
cd XianzhiKrylov
export XK_ROOT="$(pwd)"
```

`XK_ROOT` now holds the full package path. Use the same terminal for the following commands. Check the extracted files:

```
ls "$XK_ROOT/include/XianzhiKrylovAPI.h"
ls "$XK_ROOT/lib/libXianzhiKrylov.so"
```

## 2.3 Place the license

Copy your downloaded license into `XianzhiKrylov/` and name it `license.key`. You can use a file manager, or run the following commands from the directory containing the license. Replace `YOUR_LICENSE.key` with its actual filename:

```
cp YOUR_LICENSE.key "$XK_ROOT/license.key"
ls "$XK_ROOT/license.key"
```

## 2.4 Build and run

```
cd "$XK_ROOT/examples/samples/17_fgmres_ilu0"
make
make run
```

`make` builds a program named `sample`. `makerun` runs it with the local `config.json` and the shared matrix file. The program prepares $A$, $b$ and the initial $x$ in CPU memory. The library handles the data preparation needed for GPU computation.

A successful run prints solver statistics and an independent residual check:

```
XianzhiKrylov status=1 converged=1 iterations=...
Independent residual RHS 1: relative_l2=... absmax=... PASS
```

The dots stand for actual values from the run. `PASS` means that the sample checked the result using the original $A$ and $b$. Immediately after running the program, enter `echo$?`. An output of `0` means that the program completed successfully.

## 2.5 Runtime library paths

The sample Makefile locates XianzhiKrylov automatically. If a toolkit shared library is missing at runtime, run this in the sample directory:

```
ldd ./sample
```

A line containing `notfound` names a missing library. Pass the directory containing that library as `GPU_RUNTIME_LIB_DIR`. For example, if the file is installed in `/usr/local/cuda-12.4/lib64`:

```
make run GPU_RUNTIME_LIB_DIR=/usr/local/cuda-12.4/lib64
```

Use the actual directory for your installation. This setting helps the program find its runtime dependencies.

After a successful run, remove the compiled program when you no longer need it:

```
make clean
```

## 2.6 Using the GitHub samples

The repository is solved-tech-dev/xianzhikrylov-samples. On the page, select `Code`, then `DownloadZIP`. Extract the ZIP and open the resulting `xianzhikrylov-samples-main/` directory. Extract the library package under that directory's `libraries/` folder, so that its headers are under `libraries/XianzhiKrylov/include/` and its license is `libraries/XianzhiKrylov/license.key`. The same `make` and `makerun` commands then work in a sample directory. For example, open a terminal in the extracted repository and run:

```
cd 17_fgmres_ilu0
```

```
make
make run
```

If the package is already elsewhere, use the path set earlier:

```
make LIBRARY_ROOT="$XK_ROOT"
make run LIBRARY_ROOT="$XK_ROOT"
```

## 2.7 Build your own C program

Include `XianzhiKrylovAPI.h` in your C source file. Link the program to `libXianzhiKrylov.so`. From the directory containing your source file, run:

```
cc -std=c11 my_app.c -o my_app \
  -I"$XK_ROOT/include" -L"$XK_ROOT/lib" \
  -Wl,-rpath,"$XK_ROOT/lib" -Wl,--allow-shlib-undefined \
  -lXianzhiKrylov -lm
./my_app
```

The output program is named `my_app`.

- `-I`: the header directory.
- `-L`: the directory containing the library.
- `-lXianzhiKrylov`: the library to link.
- `-rpath`: the runtime search path for this library.

Keep the license in the package root selected by `XK_ROOT`. A relative configuration path passed to `config_set()` is relative to the directory from which you run the program. If toolkit libraries need an extra search path, add their actual directory to `LD_LIBRARY_PATH` before running. The previous section's `GPU_RUNTIME_LIB_DIR` is a variable used by the sample Makefile.

# 3 Calling the Library

## 3.1 Compute and data objects

An application creates three kinds of object:

- A compute object, `handle`, stores the device number, configuration, prepared data and statistics. It is also called a handle.
- A matrix object, `matrix`, records the size, format and array locations of $A$.
- Vector objects record the array locations of $b$ and $x$.

Creation functions return these objects. Keep them and pass them to later calls.

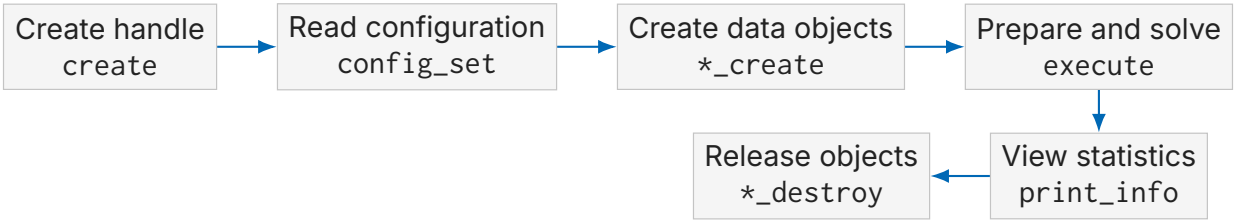



Figure 1: Calling sequence for one solve.

## 3.2 One call or separate phases

The arguments of `xianzhikrylov_execute()` are the handle, phase, $A$, $x$ and $b$, in that order. For a first run, choose `XIANZHIKRYLOV_PHASE_ALL` to prepare and solve in one call.

| Phase | Work performed |
|---|---|
| `XIANZHIKRYLOV_PHASE_ANALYSIS` | Analyze the locations of matrix entries. Pass `NULL` for $x$ and $b$. |
| `XIANZHIKRYLOV_PHASE_NUMERIC_PREPARE` | Prepare the preconditioner using matrix values. Pass `NULL` for $x$ and $b$. |
| `XIANZHIKRYLOV_PHASE_SOLVE` | Solve using prepared data. Pass $A$, $x$ and $b$. |
| `XIANZHIKRYLOV_PHASE_ALL` | Run all three phases and write the solution into $x$. |

Successful analysis and numeric preparation return `SUCCESS`. A solve that meets the stopping condition returns `CONVERGED`. For multiple right-hand sides, all solutions must meet the condition.

## 3.3 Keeping and releasing arrays

The simplest approach is to keep $A$, $b$ and $x$ in CPU memory and select `XIANZHIKRYLOV_HOST`. The library prepares data for GPU computation and writes the final solution to your $x$ array before returning.

NVIDIA and Hygon packages also accept GPU arrays with `XIANZHIKRYLOV_DEVICE`. The arrays must be on the GPU selected by the handle. The library performs the data organization required by the chosen method.

Matrix and vector objects record array addresses; the application still owns the arrays. Keep them valid while using the objects. Finish the solve and destroy the data objects before freeing their arrays. Destroying the compute handle releases the library's computation workspace.

## 3.4 Reusing prepared data

For repeated solves, use these rules:

| What changes next | Phases to run |
|---|---|
| Only $b$ or the initial $x$ | `SOLVE`. |
| Only values in $A$; entry locations stay the same | `NUMERIC_PREPARE`, then `SOLVE`. |
| The structure of $A$, or the matrix object | `ANALYSIS`, `NUMERIC_PREPARE`, then `SOLVE`. |
| Perform all work in one call | `ALL`. |

For value updates, keep the matrix object and array addresses, and write new values into the existing arrays. To change configuration, create a new compute handle and load the new file. Call one handle sequentially. For independent systems running concurrently, use separate compute and data objects.

## 3.5 Reading timings

`xianzhikrylov_print_info()` prints iteration counts, residuals and timings for the latest solve. `analysis_seconds`, `numeric_prepare_seconds` and `solve_seconds` are the phase times, in seconds. `prepare_solve_seconds` is the total preparation and solve time actually used for this solve.

File reading, object creation, CPU–GPU data copies, printing and destruction are outside this total. With separate phase calls, application waiting time between calls is also excluded. Repeating only `SOLVE` gives zero analysis and numeric preparation time for that solve. Multiple-RHS solve time includes every right-hand side.

For external AMG, `amg_setup_seconds` records the latest AMG preparation time. `amg_apply_seconds` and `amg_apply_calls` record AMG time and call count for the current solve. These are components of the total, useful for understanding where time is spent.

# 4 Data Types and Storage

This chapter explains how to pass your own $A$ and $b$ to the library. `XianzhiKrylovData.h` defines data types; `XianzhiKrylovAPI.h` declares creation and solver functions. Array indices start at zero.

## 4.1 Precision and C types

| Type constant | C array element | Meaning |
|---|---|---|
| `XIANZHIKRYLOV_FP64` | `double` | Double-precision real number. |
| `XIANZHIKRYLOV_FP32` | `float` | Single-precision real number. |
| `XIANZHIKRYLOV_COMPLEX_FP64` | `xianzhikrylov_complex_t` | `real` and `imag` are each a `double`. |
| `XIANZHIKRYLOV_COMPLEX_FP32` | `xianzhikrylov_complex_fp32_t` | `real` and `imag` are each a `float`. |
| `XIANZHIKRYLOV_DD` | High and low `double` arrays | Their sum represents one high-precision real number. |

DD stands for Double-Double and provides about 106 significant binary bits. See Appendix A for the single-RHS precision samples.

## 4.2 Describing a matrix

`xianzhikrylov_matrix_data_t` records the format, precision, dimensions, indices and value arrays. Each array is described by `data`, its address, and `bytes`, its size in bytes. Start with a zeroed structure and set its version and size from the header:

```
xianzhikrylov_matrix_data_t data = {0};
data.abi_version = XIANZHIKRYLOV_ABI_VERSION;
data.struct_size = sizeof(data);
```

Keep these initialization statements as shown. Fill the fields needed by the selected matrix format; leave other array fields zero.

## 4.3 CSR: store entries row by row

CSR uses three arrays:

- `row_offsets`: `int32_t`, with $n + 1$ elements. It marks the start and end of each row in the entry arrays. Its first element is 0; its last is the entry count `nnz`.
- `column_indices`: `int32_t`, with `nnz` elements. It gives the column of each entry. Columns in each row are strictly increasing.
- `values`: `nnz` matrix values, in the same order as the column indices. Each row includes its diagonal entry.

For example:

$$A = \begin{bmatrix} 4 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 3 \end{bmatrix}, \qquad b = \begin{bmatrix} 3 \\ 2 \\ 2 \end{bmatrix}.$$

The following fragment creates this matrix and two vectors. The initial guess is zero; the solution should be close to the all-ones vector.

```
int32_t row_offsets[] = {0, 2, 5, 7};
int32_t column_indices[] = {0, 1, 0, 1, 2, 1, 2};
double values[] = {4, -1, -1, 4, -1, -1, 3};
double b[] = {3, 2, 2};
double x[] = {0, 0, 0};

xianzhikrylov_matrix_data_t data = {0};
data.abi_version = XIANZHIKRYLOV_ABI_VERSION;
data.struct_size = sizeof(data);
data.format = XIANZHIKRYLOV_CSR;
data.value_type = XIANZHIKRYLOV_FP64;
data.index_type = XIANZHIKRYLOV_INDEX_I32;
data.block_size = 1;
data.block_layout = XIANZHIKRYLOV_BLOCK_ROW_MAJOR;
data.memory_space = XIANZHIKRYLOV_HOST;
data.device_id = -1;
data.rows = 3;
data.columns = 3;
data.entries = 7;
data.row_offsets.data = row_offsets;
data.row_offsets.bytes = sizeof(row_offsets);
data.column_indices.data = column_indices;
data.column_indices.bytes = sizeof(column_indices);
data.values.data = values;
data.values.bytes = sizeof(values);

xianzhikrylov_matrix_t matrix = NULL;
xianzhikrylov_vector_t rhs = NULL, solution = NULL;
xianzhikrylov_status_t status;

status = xianzhikrylov_matrix_create(&matrix, &data);
if (status != XIANZHIKRYLOV_SUCCESS) goto cleanup;
status = xianzhikrylov_vector_create(
    &rhs, 3, b, XIANZHIKRYLOV_FP64, XIANZHIKRYLOV_HOST);
if (status != XIANZHIKRYLOV_SUCCESS) goto cleanup;
status = xianzhikrylov_vector_create(
    &solution, 3, x, XIANZHIKRYLOV_FP64, XIANZHIKRYLOV_HOST);
if (status != XIANZHIKRYLOV_SUCCESS) goto cleanup;
```

This is a data-creation fragment. `cleanup` is the application's resource-release label. A complete program also creates a compute handle, loads a configuration, calls `execute()` and releases the objects. Chapter 6 includes the complete packaged program. These arrays are in CPU memory, so the matrix descriptor's `device_id` is `-1`. The GPU used for computation is selected by `xianzhikrylov_create()`.

## 4.4 Other sparse formats

### 4.4.1 BSR: store small dense blocks

BSR groups neighboring entries into small square blocks. `rows`, `columns` and `entries` count block rows, block columns and nonzero blocks. `block_size` gives the block dimension;

each BSR4 block has $4\times4$ entries. Block row offsets and block column indices use `uint64_t` arrays with `XIANZHIKRYLOV_INDEX_I64`. `diagonal_positions` records the position of each diagonal block in the block arrays. Store values block by block, with each block in row-major order, using `XIANZHIKRYLOV_BLOCK_ROW_MAJOR`. The vector length is the block row count multiplied by the block size. See `26_bsr4_fgmres_ilu0`.

### 4.4.2 MSR: separate diagonal and off-diagonal values

MSR uses `values` for diagonal entries and `offdiagonal_values` for off-diagonal entries. The `int32_t` arrays `row_offsets` and `column_indices` describe off-diagonal connections; `entries` counts these connections.

For block size $s$, `rows` and `columns` count blocks. Each diagonal block has $s^2$ values in row-major order. Each off-diagonal connection stores one scalar, representing that scalar times an $s \times s$ identity matrix. $s = 1$ is scalar MSR; $s = 3$ describes this form of three-component system. See the scalar example `29_msr_bicgstab_jacobi`.

### 4.4.3 LDU: separate diagonal, lower and upper values

`values` stores the diagonal. For a connection between rows $i < j$, `lower_address` stores $i$ and `upper_address` stores $j$. `lower_values` stores $A_{ji}$; `upper_values` stores $A_{ij}$. Addresses use `int32_t`. `entries` counts connections, and `rows` counts scalar rows. This form is common in finite-volume applications. See `30_ldu_pcg_dic` and `31_ldu_bicgstab_dilu`.

## 4.5 One vector and multiple vectors

Use `xianzhikrylov_vector_create()` for one contiguous vector. For multiple right-hand sides, use `xianzhikrylov_multiple_vector_create()` and specify the vector length `length`, vector count `rhs_count`, array stride `leading_dimension`, and layout:

| Layout | Element $i$ of vector $j$ |
|---|---|
| `XIANZHIKRYLOV_RHS_MAJOR` | Store one vector after another: `data[j*leading_dimension+i]`. The stride is at least `length`. |
| `XIANZHIKRYLOV_ROW_MAJOR` | Store all vectors' entries for one row together: `data[i*leading_dimension+j]`. The stride is at least `rhs_count`. |

Strides count elements, not bytes. Ordinary methods accept 1–10 right-hand sides; external AMG methods handle one at a time. $A$, $b$ and $x$ must have the same scalar row count; $b$ and $x$ must have the same vector count.

## 4.6 DD input

DD matrix input uses CSR. `values` and `values_low` hold the high and low arrays. Each vector also uses two arrays; a number is the sum of its two parts. For one DD vector, call `xianzhikrylov_multiple_vector_create()` with `rhs_count=1`, `data` pointing to the high

array and `low` to the low array. This is still a single right-hand side; the function is used to pass both numeric arrays. See `35_dd_fgmres`.

# 5 Function Reference

The public C API has ten functions. All return `xianzhikrylov_status_t`. `XIANZHIKRYLOV_SUCCESS` means an operation completed; `XIANZHIKRYLOV_CONVERGED` means a solve met its stopping condition. Chapter 7 explains the status codes.

## 5.1 xianzhikrylov_create()

```
xianzhikrylov_status_t xianzhikrylov_create(
    int32_t device_id,
    xianzhikrylov_handle_t *handle);
```

| Parameter | Direction | Meaning |
|---|---|---|
| `device_id` | Input | GPU number visible to the current process, usually 0. |
| `handle` | Output | Receives the handle on success; NULL on failure. |

Create a compute handle for a GPU.

## 5.2 xianzhikrylov_config_set()

```
xianzhikrylov_status_t xianzhikrylov_config_set(
    xianzhikrylov_handle_t handle,
    xianzhikrylov_config_option_t option,
    const char *value);
```

| Parameter | Direction | Meaning |
|---|---|---|
| `handle` | Input | A successfully created compute handle. |
| `option` | Input | Use `XIANZHIKRYLOV_CONFIG_FILE`. |
| `value` | Input | A null-terminated string containing the JSON file path. |

Load a JSON configuration and perform the configuration-time license check. Call before the first execute that uses the configuration. The library copies the path string.

## 5.3 xianzhikrylov_matrix_create()

```
xianzhikrylov_status_t xianzhikrylov_matrix_create(
    xianzhikrylov_matrix_t *matrix,
    const xianzhikrylov_matrix_data_t *data);
```

| Parameter | Direction | Meaning |
|---|---|---|
| matrix | Output | Receives the matrix object. |
| data | Input | Matrix format, precision, dimensions and array information; see Chapter 4. |

Create an object describing A. Numerical preparation happens during execute. Keep the referenced arrays valid while using the object.

## 5.4 xianzhikrylov_vector_create()

```
xianzhikrylov_status_t xianzhikrylov_vector_create(
    xianzhikrylov_vector_t *vector,
    int64_t size,
    void *data,
    xianzhikrylov_value_type_t value_type,
    xianzhikrylov_memory_space_t memory_space);
```

| Parameter | Direction | Meaning |
|---|---|---|
| vector | Output | Receives the vector object. |
| size | Input | Number of elements, equal to the scalar matrix order. |
| data | In / out | Array address. The library writes the result here when this is the solution object. |
| value_type | Input | Numeric type of the array elements. |
| memory_space | Input | XIANZHIKRYLOV_HOST or XIANZHIKRYLOV_DEVICE. |

Create one contiguous vector. For a DD vector, use multiple_vector_create with rhs_count=1 to pass high and low arrays.

## 5.5 xianzhikrylov_multiple_vector_create()

```
xianzhikrylov_status_t xianzhikrylov_multiple_vector_create(
    xianzhikrylov_vector_t *vector,
    int64_t length,
    int64_t rhs_count,
    int64_t leading_dimension,
    xianzhikrylov_layout_t layout,
    void *data,
    void *low,
    xianzhikrylov_value_type_t value_type,
    xianzhikrylov_memory_space_t memory_space);
```

| Parameter | Direction | Meaning |
|---|---|---|
| vector | Output | Receives the vector object. |
| length | Input | Elements per vector. |
| rhs_count | Input | 1–10 for ordinary methods; 1 for external AMG methods. |
| leading_dimension | Input | Distance between columns or rows, in complete numeric elements. |
| layout | Input | XIANZHIKRYLOV_RHS_MAJOR or XIANZHIKRYLOV_ROW_MAJOR. |
| data | In / out | Data array; the high array for DD. |
| low | In / out | DD low array; NULL for other types. |
| value_type | Input | Numeric type. |
| memory_space | Input | XIANZHIKRYLOV_HOST or XIANZHIKRYLOV_DEVICE. |

Create one or more vectors with an explicit layout. This function also accepts the two arrays of a single DD vector.

## 5.6 xianzhikrylov_execute()

```
xianzhikrylov_status_t xianzhikrylov_execute(
    xianzhikrylov_handle_t handle,
    xianzhikrylov_phase_t phase,
    xianzhikrylov_matrix_t matrix,
    xianzhikrylov_vector_t solution,
    xianzhikrylov_vector_t rhs);
```

| Parameter | Direction | Meaning |
|---|---|---|
| handle | Input | A successfully configured compute handle. |
| phase | Input | Analysis, numeric preparation, solve, or all phases. |
| matrix | Input | The matrix A object. |
| solution | In / out | NULL for analysis/preparation; otherwise the initial and final x. |
| rhs | Input | NULL for analysis/preparation; otherwise the vector b object. |

Execute the chosen phase. ANALYSIS and NUMERIC_PREPARE return SUCCESS when complete. SOLVE and ALL return CONVERGED when every right-hand side meets its stopping condition. See Chapter 3 for phase names and reuse rules.

## 5.7 xianzhikrylov_print_info()

```
xianzhikrylov_status_t xianzhikrylov_print_info(
    xianzhikrylov_handle_t handle);
```

| Parameter | Direction | Meaning |
|---|---|---|
| handle | Input | The compute handle used for the solve. |

Print statistics for the latest solve: per-RHS iteration counts, residuals, RHS norms, method number and preparation/solve time. Printing does not rerun the solve.

## 5.8 xianzhikrylov_matrix_destroy()

```
xianzhikrylov_status_t xianzhikrylov_matrix_destroy(
    xianzhikrylov_matrix_t matrix);
```

| Parameter | Direction | Meaning |
|---|---|---|
| matrix | Input | Matrix object to release. |

Release the matrix object. Its arrays remain owned by the application. Set the application's object variable to NULL after success.

## 5.9 xianzhikrylov_vector_destroy()

```
xianzhikrylov_status_t xianzhikrylov_vector_destroy(
    xianzhikrylov_vector_t vector);
```

| Parameter | Direction | Meaning |
|---|---|---|
| vector | Input | Vector object to release. |

Release a single-vector or multiple-vector object. Its arrays remain owned by the application. Set the application's object variable to NULL after success.

## 5.10 xianzhikrylov_destroy()

```
xianzhikrylov_status_t xianzhikrylov_destroy(
    xianzhikrylov_handle_t handle);
```

| Parameter | Direction | Meaning |
|---|---|---|
| handle | Input | Compute handle to release. |

Release the compute handle, configuration, workspace, prepared data and external AMG objects. Set the application's handle variable to NULL after success.

# 6 Samples and Configuration

## 6.1 Start with a packaged sample

`17_fgmres_ilu0` uses FGMRES with an ILU(0) preconditioner to solve a general nonsymmetric CSR system with one million rows. After setting `XK_ROOT` in Chapter 2, use the same terminal:

```
cd "$XK_ROOT/examples/samples/17_fgmres_ilu0"
make
make run
```

The local `sample.c` is a complete program. It uses helpers from `common/sample_common.h`:

| Helper | Purpose |
|---|---|
| `xk_matrix_load` | Read the matrix pattern and generate values and $b$. |
| `xk_binding_make` | Fill the matrix description and call the creation functions for $A$ and $b$. |
| `xk_solution_init` | Allocate a zeroed $x$ array and create its vector object. |
| `xk_verify` | Independently compute the residual from the original $A$, $b$ and returned $x$. |
| `xk_*_free` | Destroy data objects and free arrays allocated by the sample. |

These helpers are sample code, available for reading and reuse. The following `sample.c` matches the repository. Build it together with the shared header and matrix file described above.

```
/* XianzhiKrylov sample 17_fgmres_ilu0: fgmres / ilu0. */
#include "sample_common.h"
#include <stdio.h>

int main(int argc, char **argv)
{
    const char *matrix_path = argc > 1 ? argv[1] : "../matrices/transport_1m.csrpat";
    const char *json_path = argc > 2 ? argv[2] : "config.json";
    xk_matrix matrix = {0};
    xk_binding binding = {0};
    xk_solution solution = {0};
    xianzhikrylov_handle_t handle = NULL;
    xianzhikrylov_status_t status;
    int rc = 2, destroyed = 0;

    if (!xk_matrix_load(&matrix, matrix_path))
        goto done;
    if (!xk_binding_make(&binding, &matrix, XIANZHIKRYLOV_CSR, 1)) {
        fprintf(stderr, "cannot prepare matrix format\n");
        goto done;
    }
    if (!xk_solution_init(&solution, &matrix)) {
        fprintf(stderr, "cannot allocate solution\n");
        goto done;
    }

    status = xianzhikrylov_create(0, &handle);
    if (status != XIANZHIKRYLOV_SUCCESS) {
```

```
        fprintf(stderr, "create status=%d\n", (int)status);
        rc = 1;
        goto done;
    }
    status = xianzhikrylov_config_set(
        handle, XIANZHIKRYLOV_CONFIG_FILE, json_path);
    if (status != XIANZHIKRYLOV_SUCCESS) {
        fprintf(stderr, "config_set status=%d\n", (int)status);
        rc = 1;
        goto done;
    }
    /* HOST A, b, and x are prepared by the library during execute(ALL). */
    status = xianzhikrylov_execute(
        handle, XIANZHIKRYLOV_PHASE_ALL, binding.matrix_object,
        solution.vector_object, binding.rhs_object);
    if (status != XIANZHIKRYLOV_CONVERGED) {
        fprintf(stderr, "execute status=%d (not converged)\n", (int)status);
        rc = 1;
        goto done;
    }
    {
        const xianzhikrylov_status_t print_status =
            xianzhikrylov_print_info(handle);
        const xianzhikrylov_status_t destroy_status =
            xianzhikrylov_destroy(handle);
        handle = NULL;
        const int residual_ok = xk_verify(&matrix, &solution, 1e-8L);
        destroyed = 1;
        rc = print_status == XIANZHIKRYLOV_SUCCESS &&
             destroy_status == XIANZHIKRYLOV_SUCCESS &&
             residual_ok ? 0 : 1;
    }

done:
    if (handle && !destroyed)
        (void)xianzhikrylov_destroy(handle);
    xk_solution_free(&solution);
    xk_binding_free(&binding);
    xk_matrix_free(&matrix);
    return rc;
}
```

## 6.2 The sample configuration

This is the complete `17_fgmres_ilu0/config.json`:

```
{
  "schema_version": 1,
  "matrix_format": "csr",
  "block_size": 1,
  "solver": {
    "method": "fgmres",
    "preconditioner": "ilu0",
    "max_iterations": 2000,
    "restart": 40,
    "relative_tolerance": 1e-9,
    "absolute_tolerance": 0,
    "initial_guess": "zero"
  }
}
```

| Field | Value or meaning |
|---|---|
| `schema_version` | Use 1 for ordinary solver configurations. |
| `matrix_format` | `auto`, `csr`, `bsr`, `msr`, or `ldu`. `auto` takes the format from the matrix object. |
| `matrix_symmetry` | `general`, `symmetric`, or `spd` for a symmetric positive-definite matrix. |
| `block_size` | Use 1 for scalar CSR, or the actual block dimension. |
| `solver.method` | Iterative method name; see the following table. |
| `solver.preconditioner` | Preconditioner name; see the following table. |
| `solver.max_iterations` | Maximum iteration count. Reaching it returns the corresponding status. |
| `solver.restart` | Number of iteration vectors kept per cycle in GMRES, FGMRES or GCR. Larger values use more memory. |
| `solver.relative_tolerance` | Relative residual threshold. A smaller value is stricter. |
| `solver.absolute_tolerance` | Absolute residual threshold. See Chapter 7 for its relation to relative tolerance. |
| `solver.initial_guess` | `zero`, `provided`, or `reuse`; explained below. |

## 6.3 Choose a method and preconditioner

Start with a sample that matches your matrix, then adjust tolerances and iteration limits in its complete configuration. CG, PCG and FCG require a symmetric positive-definite matrix. IC(0) also requires a symmetric positive-definite matrix suitable for incomplete Cholesky factorization. For a general nonsymmetric matrix, FGMRES with ILU(0) is a useful starting point.

| Format | Preconditioner value | Method values |
|---|---|---|
| CSR | none or `identity` | `cg`, `bicg`, `bicgstab`, `gmres`, `fgmres`, `gcr`. |
| CSR | `jacobi` | `pcg`, `fcg`, `bicg`, `bicgstab`, `gmres`, `fgmres`, `gcr`. |
| CSR | `ilu0` | `bicg`, `bicgstab`, `gmres`, `fgmres`, `gcr`. |
| CSR | `ic0` | `pcg`, `fcg`, `bicg`, `bicgstab`, `gmres`, `fgmres`, `gcr`. |
| BSR4 | `ilu0`, `block_jacobi`, `multicolor_block_dilu` | `fgmres`. |
| MSR | `jacobi` | `bicgstab`. |
| LDU | `dic` | `pcg`. |
| LDU | `dilu` | `bicgstab`. |

The NVIDIA and Hygon samples provide complete examples of these combinations. The BSR4 ILU(0) sample also sets `kernels.factor=bsr_native_ilu0` and `kernels.sptrsv=bsr_jacobi4`. Keep these settings when using that sample.

## 6.4 Separate phases and data updates

The following is a calling fragment. Check each returned status and proceed only after success:

```
status = xianzhikrylov_execute(
    handle, XIANZHIKRYLOV_PHASE_ANALYSIS,
    matrix, NULL, NULL);
if (status != XIANZHIKRYLOV_SUCCESS) goto cleanup;
status = xianzhikrylov_execute(
    handle, XIANZHIKRYLOV_PHASE_NUMERIC_PREPARE,
    matrix, NULL, NULL);
if (status != XIANZHIKRYLOV_SUCCESS) goto cleanup;
status = xianzhikrylov_execute(
    handle, XIANZHIKRYLOV_PHASE_SOLVE,
    matrix, solution, rhs);
if (status != XIANZHIKRYLOV_CONVERGED) goto cleanup;
```

When only $b$ changes, update its array and run `SOLVE` again. When only the values of $A$ change, run `NUMERIC_PREPARE`, then `SOLVE`. See `36_multiple_rhs_reuse/sample.c` for complete status checks and reuse.

## 6.5 Initial guess

Set `"initial_guess":"zero"` in the complete configuration's `solver` object to start from zero. If you have an approximate solution, write it into $x$ and set this field to `provided`. `reuse` also reads the application's current $x$. Keeping the previous solution therefore uses it as the next initial guess.

This section describes CSR methods that use the `solver` configuration object. Precision samples use their own `extended_solver` configurations and run with the zero initial guess in their supplied files. BSR4 FGMRES with ILU(0) accepts a nonzero initial guess with the `bsr_jacobi4` configuration above. The other packaged BSR samples and external AMG methods use a zero initial guess.

## 6.6 Learn each precision separately

`32_complex_fgmres`, `33_float_fgmres_jacobi`, `34_float_complex_fgmres` and `35_dd_fgmres` each solve one right-hand side. They use complex double, real single, complex single and DD precision for $A$, $b$ and $x$.

Use the complete `config.json` in each directory. Its `extended_solver.method` field selects the corresponding method. The word `batch` is part of the library's complex-method configuration name; sample 32 sets `batch` to 1.

## 6.7 Multiple right-hand sides: a double sample

`36_multiple_rhs_reuse` uses `double` for $A$, $b$ and $x$. Three right-hand sides share a matrix and its prepared preconditioner. For example, store three length-$n$ vectors one after another:

```
status = xianzhikrylov_multiple_vector_create(
    &rhs, n, 3, n, XIANZHIKRYLOV_RHS_MAJOR,
    b, NULL, XIANZHIKRYLOV_FP64, XIANZHIKRYLOV_HOST);
```

```
if (status != XIANZHIKRYLOV_SUCCESS) goto cleanup;
status = xianzhikrylov_multiple_vector_create(
    &solution, n, 3, n, XIANZHIKRYLOV_RHS_MAJOR,
    x, NULL, XIANZHIKRYLOV_FP64, XIANZHIKRYLOV_HOST);
if (status != XIANZHIKRYLOV_SUCCESS) goto cleanup;
```

Each array holds $3n$ doubles. Check both creation statuses, then call `execute()`. The sample updates $b$, solves again, and checks each result independently.

## 6.8 Use LinglongAMG

AMG stands for algebraic multigrid. As an external preconditioner, LinglongAMG runs one AMG cycle per preconditioner call, while XianzhiKrylov controls the outer iteration. The libraries manage separate computation objects. The application passes $A$, $b$ and $x$ through XianzhiKrylov's API.

NVIDIA and Hygon packages can use this complete configuration for an outer DD-FGMRES solver. Replace both paths with the actual LinglongAMG shared library and configuration paths:

```
{
  "extended_solver": {
    "method": "fgmres_dd",
    "provider": "linglongamg",
    "library_path": "/path/to/liblinglongamg.so",
    "config_path": "/path/to/amg_config.json",
    "restart": 15,
    "max_cycles": 10,
    "relative_tolerance": 1e-9,
    "inner_method": "none"
  }
}
```

This combination takes FP64 $A$ and $b$, returns DD $x$, and solves one right-hand side from a zero initial guess. Configure LinglongAMG for one complete cycle, for example `method=amg` and `max_iters=1`. XianzhiKrylov's `relative_tolerance` controls outer convergence.

Another method name is `projected_pcg`. It uses FP64 for $A$, $b$ and $x$, with `max_iterations` as the iteration limit. Use it for systems suited to PCG with constant-mode projection. For ordinary symmetric positive-definite systems, see the PCG samples above.

AMG preparation occurs in `NUMERIC_PREPARE`; AMG applications occur in `SOLVE`. Each library uses its own installation and license. See the LinglongAMG product manual for its installation and configuration fields.

# 7 Output, Verification and Troubleshooting

## 7.1 Check whether a solve succeeded

Object creation, configuration and similar operations return `XIANZHIKRYLOV_SUCCESS` (0) on success. A solve returns `XIANZHIKRYLOV_CONVERGED` (1) when every right-hand side meets the library's stopping condition. The samples also compute the residual from the original $A$ and $b$ and the returned $x$, then print `PASS` when that check succeeds.

| Output field | Meaning |
|---|---|
| `iterations` | Actual iteration count. Multiple-RHS statistics are printed for each right-hand side. |
| `converged` | 1 means the stopping condition was met. |
| `method` | Identifier for the method selected in the configuration file. |
| `residual` | Residual measure used by the selected method. |
| `rhs_norm` | Norm of $b$, measuring its overall size. |
| `relative_residual` | Residual norm divided by the norm of $b$. |
| `prepare_solve_seconds` | Total preparation and solve time, in seconds. |
| `relative_l2`, `absmax` | Independently computed relative L2 residual and largest absolute residual component. |

## 7.2 Residuals and tolerances

The residual is $r = b - Ax$. A small residual means that $x$ closely satisfies the original equation. The L2 norm is $\|r\|_2 = \sqrt{\sum_i |r_i|^2}$. Ordinary FP64 methods stop when

$$\|r\|_2 \le \max\bigl(atol,\ rtol\,\|b\|_2\bigr).$$

`relative_tolerance` sets $rtol$; `absolute_tolerance` sets $atol$. The larger threshold is used, so meeting either threshold is enough. The relative residual is $\|r\|_2/\|b\|_2$. When $b = 0$, use the absolute residual.

For FP32, complex FP32 and DD input methods, `tolerance` is a relative L2 tolerance; `absolute_tolerance` is an absolute L2 tolerance. External AMG methods use `relative_tolerance` for the outer relative L2 residual.

For the complex FP64 method `fgmres_complex_batch`, `tolerance` is an absolute threshold on $\max_i |r_i|$. For a complex number, $|r_i|$ is its magnitude. This method also prints `residual_linf`, `rhs_norm_l2` and `relative_residual_l2`, so both measures can be inspected.

## 7.3 Independent verification

Compute $b - Ax$ again using the original input arrays and the returned solution, then compare it with your tolerance. The sample checks are in `common/sample_common.h` or the precision sample's `sample.c`. A DD check must use both high and low arrays. For strict high-precision verification, use an independent computation with enough precision. A suc-

cessful API operation and an accurate solution are distinct checks; applications should perform both.

## 7.4 Return codes

| Value | Constant | Meaning and next step |
|---|---|---|
| 0 | XIANZHIKRYLOV_SUCCESS | Operation completed. |
| 1 | XIANZHIKRYLOV_CONVERGED | All right-hand sides met the stopping condition. |
| 2 | XIANZHIKRYLOV_MAX_ITERATIONS | Iteration limit reached. Check tolerances, method and preconditioner. |
| 3 | XIANZHIKRYLOV_BREAKDOWN | Iteration could not continue. Read the diagnostic and check matrix properties. |
| 4 | XIANZHIKRYLOV_INVALID_ARGUMENT | Check arguments, dimensions, buffer sizes and initialization fields. |
| 5 | XIANZHIKRYLOV_UNSUPPORTED | Choose a format, precision, method and memory-location combination shown by this package's samples. |
| 6 | XIANZHIKRYLOV_STALE_SYSTEM | Data changed. Repeat the preparation phases described in Chapter 3. |
| 7 | XIANZHIKRYLOV_DEVICE_ERROR | Check GPU status and the toolkit environment. |
| 8 | XIANZHIKRYLOV_STATE_ERROR | Check the create, configure, prepare and solve sequence. |
| 9 | XIANZHIKRYLOV_RESIDUAL_MISMATCH | Check the original-system residual and tolerance. |
| 10 | XIANZHIKRYLOV_STAGNATED | Residual stopped decreasing. Check method and preconditioner suitability. |
| 11 | XIANZHIKRYLOV_LICENSE_ERROR | Check the license file, device, expiry and package. |

Resolve a failed call before continuing with dependent steps. Treat statuses 2, 3, 9, 10 and 11 as failures and keep their diagnostics.

## 7.5 Common problems

| Symptom | Next step |
|---|---|
| Missing `XianzhiKrylovAPI.h` | Check that `LIBRARY_ROOT` points to the package directory containing `include/` and `lib/`. |
| Missing shared library at runtime | Run `ldd ./sample`; set the toolkit library directory as in Chapter 2. |
| `config_set` returns 4 | Check the JSON path and the file's brackets and commas. |
| Configuration or solve returns 11 | Check the root-level `license.key`, its expiry and the GPU for which it was issued. |
| `matrix_create` returns 4 | Check dimensions, index type, array lengths and byte counts against Chapter 4. |
| Vector creation returns 4 | Check length and numeric type; also count, stride and layout for multiple vectors. |
| Solve does not converge | Check $A$, $b$, initial guess, tolerances, iteration limit and method suitability. |
| Changes to code are not reflected | Run `makeclean`, then `make` and `makerun`. |

# Appendix A Samples and Matrix Files

## A.1 Choose by method

Each sample directory has `sample.c`, `config.json`, `Makefile` and `README.md`. The following method samples use FP64 (`double`):

| Numbers | Methods | Matrix format |
|---|---|---|
| 01–06 | CG, BiCG, BiCGStab, GMRES, FGMRES, GCR | CSR, without a preconditioner. |
| 07–13 | PCG, FCG, BiCG, BiCGStab, GMRES, FGMRES, GCR with Jacobi | CSR. |
| 14–18 | BiCG, BiCGStab, GMRES, FGMRES, GCR with ILU(0) | CSR. |
| 19–25 | PCG, FCG, GMRES, FGMRES, BiCGStab, BiCG, GCR with IC(0) | CSR, symmetric positive-definite. |
| 26–28 | FGMRES with ILU(0), block Jacobi, or multicolor block DILU | BSR4. |
| 29 | BiCGStab with Jacobi | MSR. |
| 30–31 | PCG with DIC; BiCGStab with DILU | LDU. |

See `examples/samples/README.md` for complete directory names and commands.

## A.2 Precision samples: one right-hand side each

Use the following samples with NVIDIA and Hygon packages:

| Directory | Precision of $A$, $b$ and $x$ |
|---|---|
| `17_fgmres_ilu0` | FP64: real `double`. |
| `32_complex_fgmres` | Complex FP64: `double` real and imaginary parts. |
| `33_float_fgmres_jacobi` | FP32: real `float`. |
| `34_float_complex_fgmres` | Complex FP32: `float` real and imaginary parts. |
| `35_dd_fgmres` | DD: high and low `double` arrays. |

The DD sample uses `multiple_vector_create` to pass the high and low arrays. It sets `rhs_count=1` and solves a single system.

## A.3 Multiple right-hand sides: a separate double sample

`36_multiple_rhs_reuse` solves three FP64 right-hand sides with the same CSR matrix. It calls analysis, numeric preparation and solve separately. It then changes $b$ and solves again using the prepared matrix. Each result has an independent residual check.

## A.4 Build commands

Enter a sample directory and run:

```
make
make run
make clean
```

Run `make` in the Samples root to build all samples. Keep the relative locations of `common/`, `matrices/`, the shared build files and the sample directories.

## A.5 Matrix files and generated values

To keep downloads small, `.csrpat` files store CSR patterns and fixed generation settings. The program reads them and generates matrix values and $b$. The same file produces the same data.

| File | Order | Properties |
|---|---|---|
| `diffusion_1m.csrpat` | 1,000,000 | Symmetric positive-definite and strictly diagonally dominant. |
| `transport_1m.csrpat` | 1,000,000 | General nonsymmetric and strictly diagonally dominant. |
| `coupled4_500k.csrpat` | 500,000 | Symmetric block structure and strict diagonal dominance. |
| `complex_262k.csrpat` | 262,144 | Complex and strictly diagonally dominant; one RHS. |

Ordinary single-RHS samples construct $b = A\mathbf{1}$ using the all-ones vector, while the solver starts from the initial guess selected in the configuration. For complex data, each component of the all-ones vector is $1 + 0i$. Single-precision samples store generated data in the corresponding `float` arrays and check residuals against those arrays. The DD sample provides high and low arrays for the matrix, right-hand side and solution. The multiple-RHS sample uses $A\mathbf{1}$ and its two- and three-times multiples, then halves each right-hand side for the second solve.

### A.5.1 File layout

`XKPAT1` uses little-endian byte order. A 64-byte header is followed by `int32_trow_offsets[n+1]` and `int32_tcolumn_indices[nnz]`. The header records format version, real or complex model, order, entry count, RHS count, suggested block size, model identifier and fixed seed. See `matrices/PATTERN_FORMAT.md` for byte offsets and generation rules.

For your own matrix, fill arrays and a matrix description as shown in Chapter 4.

# Appendix B Documentation and Support

## B.1 Where to find information

| File or location | Contents |
| --- | --- |
| Package-root `README.md` | Installation and runtime requirements for the platform package. |
| `include/XianzhiKrylovAPI.h` | C function declarations and return codes. |
| `include/XianzhiKrylovData.h` | Numeric types, matrix formats and array descriptions. |
| `examples/samples/README.md` | Sample list and build/run instructions. |
| Each sample's `README.md` | Purpose, inputs and result checks. |
| Samples' `docs/` directory | Chinese and English product manuals and troubleshooting. |
| `THIRD_PARTY_NOTICES.txt` | Third-party license and attribution notices. |

## B.2 Packages and licenses

You can download library packages and licenses from:

https://store.solved-tech.com/

## B.3 Contact support

Include the package filename, GPU model, complete error output, configuration file, and the sample name or a small reproducing program. Matrix size, precision and independent residuals also help diagnose solver issues. Send licenses and business data through an agreed private channel.

Scan the following WeCom code to contact support:



# Appendix C Version History

This chapter lists the main interface changes from newest to oldest.

## XianzhiKrylov 26.44

- The public interface is now a ten-function C API and remains compatible with the 26.43 interface.
- `create` only creates the handle; JSON is loaded with `config_set`.
- Matrix, single-RHS vector and multi-RHS vector objects can be created and destroyed.
- `xianzhikrylov_execute()` runs analysis, numeric preparation, solve or the complete flow.
- The solution is written directly to the caller's solution object; there is no download or copy step.
- Matrix and vector objects use opaque types; the caller keeps ownership of the data arrays.
- The Samples and product documentation use the ten-function API.

## XianzhiKrylov 26.43

The existing 26.43 interface remains available in 26.44; this manual does not repeat its usage.

## XianzhiKrylov 26.42

- Unified developer-preview interface.
- Provided CSR, BSR, MSR, LDU and several Krylov/preconditioner combinations.